

Alagang MMC: Development Manual

System Overview

Alagang MMC is a pre-consultation coordination system built on React + Vite (frontend), Flask + SQLAlchemy (backend), and PostgreSQL 16. The kiosk is a separate React + Vite application that reads from the same backend API and runs as its own container. All three applications are containerized with Docker and served behind an nginx reverse proxy.

Repository Structure

```
a-mmc/
├── .github/workflows/           # CI: backend, frontend, kiosk workflows
├── a-mmc_backend/              # Flask API + PostgreSQL models +
migrations                      #
├── a-mmc_frontend/             # React + Vite patient/staff/admin app
├── a-mmc_kiosk/                # React + Vite kiosk (touchscreen, no
login)                          #
├── a-mmc_infra/                # Docker Compose + nginx config
├── devlogs/                    # Session logs and handoff notes (not
code)                           #
├── misc/                       # Scratch notes (not code)
└── Makefile                    # test-backend, test-frontend targets
```

Directory	Purpose
a-mmc_backend/	Flask app factory, blueprints, SQLAlchemy models, Alembic migrations, tests
a-mmc_frontend/	Patient-facing and staff-facing React SPA
a-mmc_kiosk/	Kiosk-mode React SPA (touchscreen terminal)
a-mmc_infra/	compose.yaml orchestration, nginx reverse proxy config
.github/workflows/	Three CI workflows (one per service)

Prerequisites

- Docker and Docker Compose (Docker Desktop or Docker Engine + Compose plugin)
 - Node.js 20 (matches the version used in CI and Dockerfiles)
 - Python 3.11 (matches the backend Dockerfile base image)
 - Git
-

Local Setup

1. Clone the Repository

```
git clone <repo-url>
cd a-mmc
```

2. Configure Environment Variables

```
cp a-mmc_backend/.env.example a-mmc_backend/.env
```

Edit `a-mmc_backend/.env`. The file ships with working defaults for local Docker Compose. The variables are:

Variable	Description
<code>SECRET_KEY</code>	Flask session secret. Generate with <code>python -c "import secrets; print(secrets.token_hex(32))"</code>
<code>FLASK_ENV</code>	<code>development</code> locally, <code>production</code> on Railway
<code>JWT_COOKIE_SECURE</code>	<code>false</code> locally (no HTTPS), <code>true</code> in production
<code>POSTGRES_DB</code>	Database name (<code>ammc_dev</code> by default)
<code>POSTGRES_USER</code>	PostgreSQL user (<code>postgres</code> by default)
<code>POSTGRES_PASSWORD</code>	PostgreSQL password
<code>POSTGRES_HOST</code>	Hostname of the Postgres container (<code>a-mmc-postgres</code>)

POSTGRES_PORT	5432
SQLALCHEMY_DATABASE_URI	Full connection string — set automatically from the above values
JWT_SECRET_KEY	Secret used to sign JWTs. Generate the same way as SECRET_KEY
JWT_ACCESS_TOKEN_EXPIRES	Access token TTL in seconds (3600 = 60 min)
MAIL_SERVER	SMTP host (leave blank for console preview mode)
MAIL_PORT	SMTP port (2525 for Mailtrap)
MAIL_USERNAME	SMTP username (leave blank to disable email sending)
MAIL_PASSWORD	SMTP password
MAIL_FROM	Sender address in outgoing emails
MAIL_USE_TLS	true for TLS-enabled SMTP
SYSTEM_URL	Base URL of the deployed app, used in email links

Note: If MAIL_USERNAME is left blank, the email service prints messages to stdout instead of sending. No SMTP server is needed for local development.

Note: The a-mmc_infra/.env file controls container names, image references, and ports for Docker Compose. The committed defaults are used by compose.yaml directly.

3. Start the System Stack

From the a-mmc_infra/ directory:

```
cd a-mmc_infra
docker compose up -d
```

This starts five containers:

Container	Role	Port
-----------	------	------

a-mmc-nginx	Reverse proxy — routes <code>/api/*</code> to backend, <code>/</code> to frontend	80
a-mmc-frontend	React SPA (Vite preview server)	internal only
a-mmc-backend	Flask + Gunicorn (2 workers)	5000 (internal)
a-mmc-kiosk	Kiosk React SPA (Vite preview server)	7000
a-mmc-postgres	PostgreSQL 16	5432

The backend waits for Postgres to be healthy before starting (`wait-for-postgres.sh`). The frontend and kiosk wait for the backend healthcheck to pass.

4. Run Migrations

Migrations run automatically on container start via the backend Dockerfile CMD:

```
flask db upgrade && gunicorn ...
```

To run manually:

```
docker compose exec a-mmc-backend flask db upgrade
```

To create a new migration after changing a model:

```
docker compose exec a-mmc-backend flask db migrate -m "describe change"
docker compose exec a-mmc-backend flask db upgrade
```

5. Seed the Database

Seeding is done by running `seed.py` directly inside the backend container. Three modes are available.

Dev seed (local development)

Creates two hardcoded synthetic clinicians, one test patient, one test clinician, one test secretary (linked to the test clinician), and walks through an appointment lifecycle to verify the setup:

```
docker compose exec a-mmc-backend python seed.py
```

This is idempotent — it skips records that already exist.

CSV bulk import

Imports clinicians from a CSV file in `a-mmc_backend/data/` . The files and their roles:

File	Contents	Use
<code>clinicians_anon.csv</code>	12 Rheumatology test clinicians with anonymized names, real schedule data, HMOs, and login credentials	Primary seed input for Rheumatology clinicians
<code>clinicians_synthetic.csv</code>	~52 synthetic clinicians across 21 other specialties, generated by <code>generate_clinicians.py</code>	Filler for non-Rheumatology departments
<code>clinicians_full.csv</code>	<code>clinicians_anon.csv</code> rows + <code>clinicians_synthetic.csv</code> rows combined	Used for the <code>--production</code> seed
<code>clinicians_real.csv</code>	Same 12 Rheumatology rows but with unmasked real names	Source data only — do not seed from this file; repo is public
<code>clinicians_sample.csv</code>	Two-row example showing all column names and formats	Reference only

To build `clinicians_full.csv` before a production seed:

```
cp a-mmc_backend/data/clinicians_anon.csv a-  
mmc_backend/data/clinicians_full.csv  
# append synthetic rows (skip the header line)  
tail -n +2 a-mmc_backend/data/clinicians_synthetic.csv >> a-  
mmc_backend/data/clinicians_full.csv
```

To import only the 12 Rheumatology test clinicians:

```
docker compose exec a-mmc-backend python seed.py data/clinicians_anon.csv
```

CSV import is idempotent, rows where `login_email` already exists are skipped. For each Rheumatology clinician created, a `credentials_manifest.txt` is written to `a-mmc_backend/data/`.

Production seed

Runs all three steps in sequence: CSV import, admin account creation, and Rheumatology secretary creation (one secretary per `testclinician1-12@ammc.com`). Writes a unified credentials manifest covering admin, all 12 clinicians, and all 12 secretaries.

```
docker compose exec a-mmc-backend python seed.py data/clinicians_full.csv  
--production
```

Admin credentials created by the production seed:

- **Email:** `admin@alagang-mmc.local`
- **Password:** `ChangeMe123!`

Change the admin password after first login.

There is also a one-time HTTP bootstrap endpoint (`POST /api/admin/seed-first-admin`) that creates only the admin account. It returns 403 if any admin already exists and must be removed before production. Use the `--production` seed script instead when possible.

All test accounts are lost when volumes are destroyed (`docker compose down -v`).

6. Verify the Setup

```
curl http://localhost:80/api/health
```

Expected response:

```
{"status": "ok"}
```

Running Tests

Backend

```
cd a-mmc_backend  
python -m pytest -vv --cov=app --cov-fail-under=70 tests/
```

Or via the Makefile from the repo root (CI target, which uses `--cov-fail-under=60` and does not fail the build on threshold):

```
make test-backend
```

Coverage threshold enforced by `.coveragerc` is **70%**. The following are excluded from coverage measurement:

- `app/routes/*`
- `app/seed.py`
- `app/services/email_service.py`
- `app/services/email_templates.py`
- `app/__init__.py`

Tests require a running PostgreSQL instance. The CI workflow starts one as a service container. For local runs outside Docker, set `ACTIONS_TEST_DATABASE_URL` or configure `.env` with a reachable database.

Frontend / Kiosk

No tests for these components have been made at the time of writing

Project Structure: Key Files

Backend

File	Purpose
<code>run.py</code>	Entry point. Calls <code>create_app()</code> , starts dev server on port 5000. Production uses Gunicorn directly.
<code>app/__init__.py</code>	App factory (<code>create_app()</code>). Initializes SQLAlchemy, Migrate, CORS, JWTManager, Limiter. Registers all blueprints. Defines the JWT blocklist loader and centralized error handlers.
<code>config/BaseConfig.py</code>	Shared config: JWT settings, CORS origins, limiter storage.
<code>config/DevelopmentConfig.py</code>	Extends BaseConfig. Reads from <code>.env</code> . Used locally.
<code>config/ProductionConfig.py</code>	Extends BaseConfig. Reads <code>DATABASE_URL</code> from environment (Railway injects this).

Models (`app/models/`)

File	Models
<code>clinician.py</code>	<code>Clinician</code> , <code>ClinicianSchedule</code> , <code>ClinicianHMO</code> , <code>ClinicianInfo</code> , <code>ClinicianTimeslot</code>
<code>secretary.py</code>	<code>Secretary</code> , <code>SecretaryClinicianLink</code> (M2M junction)
<code>patient.py</code>	<code>Patient</code>
<code>appointment.py</code>	<code>Appointment</code>
<code>admin.py</code>	<code>Admin</code>

Routes (`app/routes/`)

File	URL Prefix	Key endpoints
<code>auth_routes.py</code>	<code>/api/auth</code>	<code>POST /login</code> , <code>POST /logout</code> , <code>GET /me</code> , <code>POST /refresh</code> , <code>PATCH /change-password</code>

<code>clinician_routes.py</code>	<code>/api/clinicians</code>	CRUD clinicians, schedules, HMOs, infos, timeslots
<code>secretary_routes.py</code>	<code>/api/secretaries</code>	CRUD secretaries, POST/DELETE <code>/<id>/clinicians/<id></code> (link/unlink)
<code>patient_routes.py</code>	<code>/api/patients</code>	CRUD patients, registration
<code>timeslot_routes.py</code>	<code>/api/timeslots</code>	Slot generation and listing
<code>appointment_routes.py</code>	<code>/api/appointments</code>	Full appointment lifecycle (create, status update, reschedule, cancel)
<code>admin_routes.py</code>	<code>/api/admin</code>	Dashboard, analytics, user lists, create admin, email previews, seed
<code>upload_routes.py</code>	<code>/api/uploads</code>	File upload stub (not wired to storage)

Services (`app/services/`)

File	Purpose
<code>auth_service.py</code>	<code>hash_password</code> , <code>verify_password</code> , <code>get_*_by_email</code> for all 4 roles, <code>build_identity</code> , JWT blocklist (<code>blocklist_token</code> , <code>is_token_blocked</code>)
<code>appointment_service.py</code>	<code>has_overlap()</code> — checks for scheduling conflicts on a given date
<code>timeslot_service.py</code>	<code>generate_slots()</code> — idempotent, 60-day rolling window, <code>commit=False</code> . <code>regenerate_slots_for_schedule_change()</code> — called on schedule PATCH.
<code>email_service.py</code>	Notification functions for all appointment events. Console preview mode when <code>MAIL_USERNAME</code> is unset. All sends are post-commit and wrapped in try/except.
<code>email_templates.py</code>	HTML templates for all outgoing emails (confirmation, reschedule, cancellation, credentials, reminder).

Tests (`tests/`)

File	What it covers
<code>conftest.py</code>	Pytest fixtures: test app, test client, database session, test users per role
<code>test_auth_service.py</code>	<code>hash_password</code> , <code>verify_password</code> , token blocklist, <code>build_identity</code>
<code>test_appointment_service.py</code>	<code>has_overlap()</code> — overlap detection across all active statuses
<code>test_timeslot_service.py</code>	<code>generate_slots()</code> — slot generation, idempotency, schedule change regen
<code>test_validators.py</code>	<code>require_fields()</code> — missing field detection

Frontend (`a-mmc_frontend/src/`)

Pages by role

Role	Path	Component
Public	<code>/find</code>	<code>FindDoctor.jsx</code> — entry point, aware vs. unaware patient split
Public	<code>/find/triage</code>	<code>GuidedSearch.jsx</code> — 2-step triage (HMO, then symptom)
Public	<code>/doctors</code>	<code>Doctors.jsx</code> — live clinician directory with filter panel
Public	<code>/clinician/:id</code>	<code>ClinicianProfile.jsx</code> — clinician detail, book CTA
Public	<code>/book/:id</code>	<code>BookAppointment.jsx</code> — full booking form
Public	<code>/login</code>	<code>Login.jsx</code> — patient auth, <code>?redirect=</code> chain
Public	<code>/register</code>	<code>Register.jsx</code> — 3-step registration, auto-login

Patient	/dashboard	PatientDashboard.jsx — upcoming appointments, reminder banner
Patient	/dashboard/appointments	PatientAppointments.jsx — full appointment history
Patient	/dashboard/profile	UpdateProfile.jsx — patient profile edit
Staff	/staff/login	StaffLogin.jsx — role selector, staff auth
Staff	/clinician-dashboard/today	ClinicianTodayView.jsx — default landing, today's queue
Staff	/clinician-dashboard	ClinicianDashboard.jsx — full appointment inbox
Staff	/clinician-dashboard/profile	ClinicianProfileManager.jsx
Staff	/clinician-dashboard/schedule	ScheduleManager.jsx
Staff	/clinician-dashboard/change-password	ChangePassword.jsx
Admin	/admin	AdminDashboard.jsx — summary counts, recent activity
Admin	/admin/analytics	AdminAnalytics.jsx — recharts, period filter
Admin	/admin/clinicians	AdminClinicians.jsx — list, add, delete
Admin	/admin/secretaries	AdminSecretaries.jsx — list, add, link/unlink
Admin	/admin/patients	AdminPatients.jsx — list, view details
Admin	/admin/email-previews	AdminEmailPreviews.jsx — iframe preview + Copy HTML

Key components

File	Purpose
<code>components/Navbar.jsx</code>	Persistent patient navbar. Hidden on <code>/login</code> and <code>/register</code> .
<code>components/StaffLayout.jsx</code>	Staff topbar shell, wraps clinician dashboard routes.
<code>components/AdminLayout.jsx</code>	Sidebar + header shell for all <code>/admin/*</code> routes.
<code>components/shared/AppointmentDrawer.jsx</code>	C/S: centered modal (desktop) / slide-up (mobile). Patient: right drawer (desktop) / bottom (mobile).
<code>components/shared/SlotPicker.jsx</code>	Controlled date + slot picker. Mirrors backend temporal guard client-side.
<code>components/AppointmentReminderBanner.jsx</code>	Amber banner shown when an accepted appointment is tomorrow. Non-dismissible.
<code>context/AuthContext.jsx</code>	JWT in memory, silent refresh on mount, role-aware logout, <code>refreshUser()</code> .
<code>services/api.js</code>	Axios instance, request interceptor (attaches access token), 401 interceptor (retries once with refresh, guards against loop on <code>/refresh</code> itself).
<code>services/pdfService.js</code>	jsPDF A4 PDF export. Patient and staff variants. Uses <code>doc.text()</code> only.
<code>services/uploadService.js</code>	Upload stub. <code>uploadFile()</code> returns <code>null</code> until wired to storage.
<code>data/triageLogic.js</code>	Triage HMO and symptom-to-specialty mapping. Source of truth. Must be mirrored to <code>mmc_kiosk/src/data/triageLogic.js</code> on every change.

Kiosk (a-mmc_kiosk/src/)

The kiosk is a standalone React app with no routing library. Navigation is managed by a `screen` state variable in `App.jsx`.

Screen state	Component	Purpose
home	HomeScreen.jsx	Landing. Two buttons: Browse Directory, Find a Specialist.
directory	DirectoryScreen.jsx	Fetches and lists all clinicians.
clinician	ClinicianDetailScreen.jsx	Clinician profile with schedule and QR code linking to the main app's booking page.
triage	KioskTriageScreen.jsx	2-step triage flow using <code>triageLogic.js</code> (aliased as <code>@triage</code>).

The kiosk resets to `home` after 2 minutes of inactivity (`IdleTimer.jsx`, `timeoutMs={120000}`).

The `VITE_API_URL` and `VITE_MAIN_APP_URL` build arguments must be passed at Docker build time. If omitted, the bundled API base URL is `undefined`.

Architecture Notes

Authentication

- Access token: 60 minutes, returned in response body as `{ "access_token": "..." }`. Stored in React memory (not `localStorage`).
- Refresh token: 7 days, set as an `httpOnly` cookie named `refresh_token` (`samesite=Lax`).
- CSRF protection on `/refresh` only: double-submit cookie pattern. Client must send `X-CSRF-Token` header on every refresh call.
- JWT blocklist: in-memory set in `auth_service.py`. Cleared on server restart. Production should replace with Redis.
- Account lockout: 5 consecutive login failures lock an email address for 15 minutes. State is in-memory, cleared on restart.

- Token identity: `sub` field is the user ID string. Full identity (role, name, etc.) is in `additional_claims["user"]`. Always use `get_jwt()` to read claims — never re-decode the token.

Timeslots

Timeslots are pre-generated and stored in `CLINICIAN_TIMESLOT`. They are not created on demand.

- `generate_slots()` is idempotent. It generates a 60-day rolling window from the current date. It does not commit; the caller commits.
- `regenerate_slots_for_schedule_change()` is called when a clinician's schedule is saved. It replaces future slots for the affected schedule rows.
- Appointments reference a slot row by `slot_id`. Slot status (`available` | `blocked`) is written only by C/S actions or when `max_patients` is reached. Booking, cancelling, and rescheduling do not write slot status.

Email

NOTE: Email is not fully functional in production build, email outputs are verifiable via docker exec monitoring.

Eight notification functions are defined in `email_service.py`. When `MAIL_USERNAME` is not set, each function prints to stdout with the prefix `[EMAIL PREVIEW - not sent]`. No SMTP configuration is needed to run locally.

All email sends happen after a successful database commit, inside a try/except block. A failed send never causes the HTTP response to fail.

`send_noshow_confirmation_prompt()` is scaffolded but not wired to any scheduled job.

PDF Generation

`pdfService.js` uses jsPDF with `doc.text()` only. `doc.html()` is never called, which avoids the XSS vector in the bundled DOMPurify version. Two variants: patient-facing (appointment summary) and staff-facing (full appointment detail).

CSRF

Double-submit cookie is applied to `POST /api/auth/refresh` only. All other endpoints are protected by JWT bearer token requirement.

Kiosk Isolation

The kiosk is a separate Docker container built from a separate `Dockerfile`. It communicates with the backend via `VITE_API_URL`, which is set at build time through Docker build args. Container-to-container communication uses the Docker Compose internal network (`a-mmc-network`).

Deployment

Containerization

Each service has its own Dockerfile:

- `a-mmc_backend/Dockerfile`: Python 3.11 Alpine. Installs dependencies, copies source, runs `wait-for-postgres.sh` as the entrypoint, then starts with `flask db upgrade && gunicorn -b 0.0.0.0:$PORT --workers 2 --threads 1 'run:create_app()'`.
- `a-mmc_frontend/Dockerfile` (named `dockerfile`, lowercase): Node 20 Alpine. Runs `npm ci && npm run build`, then serves with `npm run preview`.
- `a-mmc_kiosk/Dockerfile`: Node 20 Alpine. Accepts `VITE_API_URL` and `VITE_MAIN_APP_URL` as build args. Runs `npm ci && npm run build`, then serves with `npm run preview`.
- `a-mmc_infra/nginx/Dockerfile`: nginx reverse proxy. Proxies `/api/*` to the backend container and `/` to the frontend container.

`compose.yaml` references images by name (set via `a-mmc_infra/.env`). To build and push updated images, build each Dockerfile individually and tag them to match the image names referenced in the Compose file.

CI/CD

Three GitHub Actions workflows trigger on push to `main` when files in the corresponding service directory change. Each also supports `workflow_dispatch` for manual runs.

Backend workflow (`.github/workflows/backend_workflow.yml`):

1. Starts a PostgreSQL 15 service container.
2. Installs Python 3.11 and runs `pip install -r requirements.txt`.
3. Runs `flask db upgrade` against the test database (`ACTIONS_TEST_DATABASE_URL`).

4. Runs `make test-backend` (pytest with coverage).
5. Builds and pushes the backend Docker image to Docker Hub (`$DOCKER_USERNAME/a-mmc-backend:latest`).

Frontend workflow (`.github/workflows/frontend_workflow.yml`):

1. Installs Node 20 and runs `npm install`.
2. Runs `make test-frontend` (`npm test --if-present`).
3. Builds and pushes the frontend Docker image to Docker Hub (`$DOCKER_USERNAME/a-mmc-frontend:latest`).

Kiosk workflow (`.github/workflows/kiosk_workflow.yml`):

1. Installs Node 20 and runs `npm install`.
2. Builds and pushes the kiosk Docker image with `VITE_API_URL` and `VITE_MAIN_APP_URL` as build args sourced from repository secrets.

Docker Hub credentials (`DOCKER_USERNAME` , `DOCKER_PASSWORD`) and the kiosk build args (`VITE_API_URL` , `VITE_MAIN_APP_URL`) must be set as GitHub repository secrets.

Environment Variables for Deployment

For production, set the following as environment variables on the host or deployment platform:

Variable	Notes
<code>SECRET_KEY</code>	Strong random string
<code>JWT_SECRET_KEY</code>	Strong random string
<code>JWT_ACCESS_TOKEN_EXPIRES</code>	<code>3600</code>
<code>JWT_COOKIE_SECURE</code>	<code>true</code> (requires HTTPS)
<code>FLASK_ENV</code>	<code>production</code>
<code>DATABASE_URL</code>	Full PostgreSQL connection string
<code>SYSTEM_URL</code>	Base URL of the deployed app
<code>MAIL_SERVER</code>	SMTP host

<code>MAIL_PORT</code>	SMTP port
<code>MAIL_USERNAME</code>	SMTP username
<code>MAIL_PASSWORD</code>	SMTP password
<code>MAIL_FROM</code>	Sender address
<code>MAIL_USE_TLS</code>	<code>true</code>

`SQLALCHEMY_DATABASE_URI` is not used in production — the `ProductionConfig` reads `DATABASE_URL` directly.

Known Constraints

The following are known limits and constraints discussed either in documentation or the final manuscript, fixes, implementations and plans to address may come in the future.

- **Single-instance only.** The JWT blocklist and account lockout state are in-memory. Multiple backend instances would not share this state. Replace with Redis before running multiple replicas.
- **Container restart clears lockout and blocklist.** Any in-progress account lockout or revoked token is lost on restart.
- **No horizontal scaling** is supported without the Redis replacement above.
- **Upload service is not wired.** `uploadService.js` returns `null` for all uploads. Profile pictures and SC/PWD ID uploads are not persisted until a storage backend is connected. Marked with `# TODO(upload)` in the codebase.
- **No-show prompt is not scheduled.** `send_noshow_confirmation_prompt()` in `appointment_routes.py` is scaffolded but not triggered by any job. Marked with `# TODO(scheduler)`.
- **jsPDF DOMPurify vulnerability.** The bundled version has a moderate XSS advisory. It is not exploitable in current usage because `doc.html()` is never called.
- **PostgreSQL connection pool** uses SQLAlchemy defaults. Tune `pool_size`, `max_overflow`, and `pool_timeout` before production under load.